

AKWABA INVEST

Feuille de route détaillée — v1.0

De zéro au Play Store — Nouveau repo from scratch

PHASE 0 Setup 1 sem.	PHASE 1 Backend Core 2 sem.	PHASE 2 Données 1 sem.	PHASE 3 Flutter MVP 4 sem.	PHASE 4 Lancement 1 sem.
-----------------------------------	--	-------------------------------------	---	---------------------------------------

Durée totale estimée	~9 semaines (disponibilité >4h/jour)
Point de départ	Nouveau repo GitHub from scratch
Backend réutilisé	Parser BOC pdfplumber, logique métier existante
Objectif final	App Android sur le Play Store — version bêta fermée

Vue d'ensemble — Timeline

Semaine	Phase	Objectif principal	Livrable clé
S1	Phase 0 Setup	Repo, VPS, PostgreSQL opérationnel	Infra prête, DB accessible
S2–S3	Phase 1 Backend Core	FastAPI + PostgreSQL + Auth + Parser BOC	API /stocks, /auth fonctionnels
S4	Phase 1 (suite) Backend Core	Portfolio, alertes, Wave Business	Paiement Premium fonctionnel
S5	Phase 2 Données	Saisie données financières 20 entreprises	Fiche action complète en DB
S6–S7	Phase 3 Flutter — Écrans core	Auth, Dashboard, Fiche action, Marché	3 écrans principaux fonctionnels
S8	Phase 3 Flutter — Premium	Portfolio, Alertes FCM, Comparateur	Fonctionnalités Premium Flutter
S9	Phase 3 Flutter — Finitions	Simulateur, UX polish, tests	App stable sur device réel
S9	Phase 4 Lancement	Play Store, bêta fermée, monitoring	APK publié, 10 bêta-testeurs

Principes de travail

1 tâche = 1 branche Git	feature/task-id — jamais coder directement sur main
1 tâche = 1 session Claude Code	Prompt précis, une seule responsabilité par session
Tests avant merge	pytest pour le backend, flutter test pour Flutter
Checkpoint obligatoire	Ne pas démarrer la phase suivante sans valider les critères
Données réelles dès S5	Toujours tester avec de vraies données BRVM, pas de mock

PHASE 0	SETUP & INFRASTRUCTURE	Repo · VPS · PostgreSQL · CI/CD	Semaine 1
---------	------------------------	---------------------------------	-----------

Objectif : avoir une infrastructure solide avant d'écrire la première ligne de code métier. Tout ce qui est posé ici ne change plus.

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P0-01	Créer le repo GitHub	Nouveau repo akwaba-invest (public ou privé). Structure : backend/ flutter/ scripts/ docs/ .github/. Initialiser avec README, .gitignore Python+Dart, LICENSE.	2h	Claude (ici)	Repo GitHub prêt
P0-02	Provisionner VPS Hetzner CX22	Commander CX22 sur hetzner.com. Ubuntu 22.04 LTS. Configurer SSH key-only (désactiver password auth). Créer user non-root avec sudo. UFW : ports 22, 80, 443 uniquement.	3h	Claude (ici)	VPS accessible SSH
P0-03	Installer PostgreSQL 15	apt install postgresql-15. Créer DB akwaba_db, user akwaba_user avec password fort. Tester connexion locale. Configurer pg_hba.conf pour connexion locale uniquement (pas d'exposition réseau).	2h	Claude (ici)	DB PostgreSQL opérationnelle
P0-04	Installer Nginx + Certbot SSL	apt install nginx certbot python3-certbot-nginx. Pointer domaine vers IP VPS. Générer certificat Let's Encrypt. Config Nginx : reverse proxy vers FastAPI port 8000. HTTPS obligatoire.	3h	Claude (ici)	HTTPS fonctionnel sur domaine
P0-05	Créer structure projet backend	Dans backend/ : app/api/routes/, app/services/, app/models/, app/schemas/, app/utlis/, tests/, alembic/. Créer pyproject.toml (poetry) ou requirements.txt. FastAPI + SQLAlchemy + Alembic + pdfplumber + firebase-admin.	2h	Claude Code	Structure dossiers + deps installées
P0-06	Créer structure projet Flutter	flutter create akwaba_invest dans flutter/. Configurer pubspec.yaml : riverpod, go_router, dio, freezed, json_serializable, flutter_secure_storage, firebase_core, firebase_messaging, fl_chart.	2h	Claude Code	flutter pub get sans erreur
P0-07	Configurer GitHub Actions CI	Workflow .github/workflows/backend.yml : sur push → pip install → pytest. Workflow flutter.yml : sur push → flutter test. Secrets GitHub : DATABASE_URL, FIREBASE_KEY.	2h	Claude Code	CI verte sur premier push
P0-08	Config Firebase projet	Créer projet Firebase akwaba-invest. Activer Authentication (Email/Password + Google). Activer Cloud Messaging (FCM). Télécharger google-services.json (Flutter) et service-account.json (backend).	1h	Manuel	Firebase Auth + FCM configurés

CHECKPOINT — Critères de passage	✓ VPS accessible via SSH depuis la machine de dev ✓ PostgreSQL répond aux requêtes locales ✓ HTTPS actif sur le domaine (certificat Let's Encrypt valide) ✓ CI GitHub Actions verte (backend + Flutter) ✓ firebase-admin SDK s'authentifie sans erreur
----------------------------------	--

PHASE 1	BACKEND CORE	FastAPI · PostgreSQL · Auth · Parser BOC · Paiements	Semaines 2–4
---------	--------------	---	--------------

Objectif : API REST complète et sécurisée. Toutes les routes que Flutter consommera sont fonctionnelles et testées avant de toucher au mobile.

1A — Schéma de base de données & migrations

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P1-01	Concevoir schéma PostgreSQL	Tables : stocks (symbol, name, sector, market_cap, float_pct, shares_outstanding), daily_prices (stock_id, date, close, volume, variation_pct), financials (stock_id, year, revenue, net_income, debt, equity), dividends (stock_id, year, gross_amount, net_amount, detachment_date, payment_date, yield_pct), users (id, email, firebase_uid, plan, created_at), subscriptions (user_id, plan, start_date, end_date, wave_ref), portfolios (user_id, name), portfolio_lines (portfolio_id, stock_id, quantity, avg_price, total_dividends_received), alerts (user_id, stock_id, type, threshold, active, last_triggered), boc_runs (date, status, stocks_parsed, error_msg).	4h	Claude (ici)	Schéma validé sur papier
P1-02	Créer modèles SQLAlchemy	Un fichier par domaine : models/stock.py, models/price.py, models/financial.py, models/dividend.py, models/user.py, models/portfolio.py, models/alert.py. Utiliser SQLAlchemy avec Field, Relationship. Pas de logique métier dans les modèles.	4h	Claude Code	Modèles SQLAlchemy sans erreur
P1-03	Générer migrations Alembic	alembic init alembic. Configurer alembic.ini avec DATABASE_URL depuis .env. alembic revision --autogenerate -m 'initial_schema'. alembic upgrade head. Vérifier tables créées dans psql.	2h	Claude Code	Tables créées en DB

1B — Authentification Firebase

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P1-04	Middleware Firebase Auth	Créer app/dependencies.py : fonction verify_firebase_token(token) qui appelle firebase_admin.auth.verify_id_token(). Retourner user_id (firebase_uid). Lever HTTPException 401 si token invalide ou expiré.	3h	Claude Code	Middleware auth fonctionnel
P1-05	Route POST /auth/register	Reçoit firebase_uid + email. Crée user en DB si inexistant (upsert). Retourne user_id interne + plan (free). Protégé par verify_firebase_token.	2h	Claude Code	Test: nouveau user créé en DB
P1-06	Route GET /auth/me	Retourne profil utilisateur connecté : id, email, plan, subscription_end_date. Protégé par verify_firebase_token.	1h	Claude Code	Test: profil retourné correctement

1C — Parser BOC & ingestion données quotidiennes

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P1-07	Porter parser pdfplumber	Copier boc_parser.py depuis ancien repo. Adapter pour écrire en PostgreSQL (SQLModel session) au lieu de Firestore. Idempotence via table boc_runs : vérifier si date déjà parsée avant insertion. Transaction atomique : tout ou rien.	5h	Claude Code	Parser écrit en PostgreSQL
P1-08	Service stock_service.py	Fonctions : get_all_stocks(), get_stock_by_symbol(symbol), get_latest_prices(limit=10), get_top_movers(n=5, direction='up' 'down'). Utiliser SQLModel select(). Typer les retours avec schemas Pydantic.	4h	Claude Code	Tests unitaires passent
P1-09	Routes GET /stocks	GET /stocks → liste toutes les actions (symbol, name, sector, close, variation_pct). GET /stocks/{symbol} → détail action + dernier cours. GET /stocks/top-movers → top 5 hausses + top 5 baisses. Pagination sur /stocks.	3h	Claude Code	Endpoints testés via Swagger UI
P1-10	Cron Linux parser BOC	Créer /etc/cron.d/akwaba-boc sur VPS : '0 18 * * 1-5 akwaba cd /app && python scripts/run_boc.py >> /var/log/akwaba_boc.log 2>&1'. Tester manuellement : python scripts/run_boc.py --date today.	2h	Claude (ici)	Cron déclenche le parser à 18h

1D — Portfolio & alertes

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P1-11	Service portfolio_service.py	add_line(user_id, stock_id, quantity, price) : calcule PRU pondéré consolidé si ligne existante (PRU = (ancien_qty*ancien_pru + new_qty*new_price) / total_qty). get_portfolio(user_id) : retourne toutes lignes avec valorisation temps réel + gain/perte + total_return (plus-value + dividendes reçus).	5h	Claude Code	PRU consolidé calculé correctement
P1-12	Routes /portfolio	GET /portfolio → liste lignes + valorisation totale. POST /portfolio/lines → ajouter ligne. DELETE /portfolio/lines/{id} → supprimer ligne. PUT /portfolio/lines/{id} → modifier quantité.	3h	Claude Code	CRUD portfolio fonctionnel
P1-13	Service alert_service.py	Logique edge-trigger : déclencher alerte seulement si price VIENT DE franchir le seuil (comparer avec last_trigger_state). Éviter spam FCM. Envoyer via firebase_admin.messaging.send(). Stocker last_triggered en DB.	4h	Claude Code	Alerte déclenchée une seule fois
P1-14	Routes /alerts	GET /alerts → liste alertes utilisateur. POST /alerts → créer alerte (stock_id, type: 'price_above' 'price_below' 'dividend', threshold). DELETE /alerts/{id}. PATCH /alerts/{id}/toggle → activer/désactiver.	3h	Claude Code	CRUD alertes fonctionnel

1E — Paiements Wave Business

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P1-15	Intégration Wave Business API	Lire doc Wave Business API. Créer service payment_service.py : create_payment_link(user_id, plan, amount) → retourne URL Wave. Webhook POST /payments/wave-webhook : vérifier signature HMAC, upgrader user vers Premium en DB, enregistrer subscription.	6h	Claude (ici) + Claude Code	Paiement test Wave → Premium activé
P1-16	Route GET /payments/plans	Retourner les plans disponibles : {mensuel: 2000 FCFA, annuel: 18000 FCFA}. Route publique (pas d'auth requise).	1h	Claude Code	Plans retournés correctement

CHECKPOINT — Critères de passage	✓ 100% des routes FastAPI documentées dans Swagger UI (/docs) ✓ pytest tests/ -v → 0 échec ✓ Parser BOC écrit correctement en PostgreSQL (test sur BOC réel) ✓ Paiement Wave test → utilisateur passe en Premium en DB ✓ Alerte push FCM reçue sur device de test
----------------------------------	---

PHASE 2	DONNÉES FINANCIÈRES	Saisie manuelle · 20 entreprises · Fiche action complète	Semaine 5
---------	---------------------	--	-----------

Objectif : alimenter la DB avec les données financières pluriannuelles des 20 entreprises prioritaires. Sans ces données, la fiche action complète (Premium) est vide.

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P2-01	Créer script d'import CSV	Script scripts/import_financials.py qui lit un CSV (symbol, year, revenue, net_income, debt, equity, shares_outstanding, dividend_gross, dividend_net, detachment_date, payment_date, float_pct) et insère en DB via SQLAlchemy. Idempotent : upsert sur (symbol, year).	3h	Claude Code	Script import fonctionnel
P2-02	Créer template CSV	Générer template CSV vide avec toutes les colonnes. Documenter chaque colonne dans README. Unités : montants en FCFA, dates en YYYY-MM-DD, pourcentages en décimal (0.085 pour 8.5%).	1h	Claude Code	Template CSV + documentation
P2-03	Collecter données 20 entreprises	Priorité : Sonatel, SGCI, Ecobank CI, BOA CI, Coris Bank, NSIA Banque, Total CI, Palmci, SOGB, SIB, BICI CI, Onatel BF, Orange CI, CFAO, Nestlé CI, SOLIBRA, SIFCA, BOA BF, Bolloré Africa, SITAB. Sources : rapports annuels PDF sur sites entreprises + BRVM. Extraire CA, résultat net, dividendes sur 3 ans minimum.	8h	Manuel + Claude (analyse PDF)	CSV rempli 20 entreprises
P2-04	Importer et vérifier données	python scripts/import_financials.py --file data/financials_2024.csv. Vérifier en DB : SELECT * FROM financials LIMIT 20. Vérifier calculs : PER = cours / (net_income / shares_outstanding). Rendement = dividend_net / cours * 100.	2h	Claude Code	20 entreprises en DB avec données vérifiées
P2-05	Endpoints fiche action complète	GET /stocks/{symbol}/financials → historique CA + résultat net 5 ans. GET /stocks/{symbol}/dividends → historique dividendes + prochain dividende. GET /stocks/{symbol}/metrics → PER, BNPA, rendement, ratio dette. GET /stocks/sector/{sector} → toutes actions d'un secteur avec métriques (comparateur).	4h	Claude Code	Fiche action complète via API
P2-06	Middleware contrôle accès Premium	Créer dependency require_premium(user) : vérifie user.plan == 'premium' ET subscription non expirée. Appliquer sur /stocks/{symbol}/financials, /stocks/{symbol}/dividends (historique complet), /stocks/sector/{sector}.	2h	Claude Code	Route Premium → 403 si non abonné

CHECKPOINT — Critères de passage	✓ 20 entreprises avec données financières 3 ans minimum en DB ✓ GET /stocks/SNTS/metrics retourne PER et rendement corrects (vérifier avec Richbourse) ✓ Route /stocks/sector/banques retourne comparateur sectoriel ✓ Accès Premium bloqué correctement pour utilisateurs gratuits
----------------------------------	---

PHASE 3	FLUTTER MVP	Android · Riverpod · GoRouter · Écrans core + Premium	Semaines 6–9
---------	-------------	---	--------------

Objectif : application Flutter Android complète, connectée à l'API, testée sur device réel. Chaque écran est construit feature par feature.

3A — Architecture Flutter & fondations (S6)

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P3-01	Architecture dossiers Flutter	lib/ : core/ (theme, router, constants, dio_client), features/ (auth/, market/, stock_detail/, portfolio/, alerts/, simulator/, comparator/, subscription/), shared/ (widgets/, models/, providers/). Un dossier par feature : data/ (repository, dto), domain/ (model, provider), presentation/ (screen, widgets).	3h	Claude (ici)	Structure dossiers créée
P3-02	Thème et design system	ThemeData avec couleurs Akwaba (vert #00875A, bleu #1A2E40, orange #F6AD55). Typography : Inter ou Roboto. Composants réutilisables : AkwabaCard, AkwabaButton, PriceChip (rouge/vert selon variation), LoadingShimmer.	4h	Claude Code	Thème appliqué sur MaterialApp
P3-03	Dio HTTP client + intercepteurs	Créer DioClient singleton : baseUrl depuis .env, timeout 30s. Intercepteur auth : injecter Firebase ID token dans header Authorization: Bearer {token}. Intercepteur erreur : gérer 401 (refresh token), 403 (afficher paywall), 500 (snackbar erreur).	3h	Claude Code	Requêtes API avec token auto-injectés
P3-04	GoRouter navigation	Routes : / (splash), /login, /register, /home, /stock/:symbol, /portfolio, /alerts, /simulator, /comparator/:sector, /subscription, /profile. Guards : redirect vers /login si non authentifié. ShellRoute pour bottom navigation bar (5 onglets).	3h	Claude Code	Navigation entre écrans fonctionnelle
P3-05	Écrans Auth (Login + Register)	Login : champ email + password, bouton Google Sign-In. Register : email + password + confirmation. Après auth Firebase : appel POST /auth/register pour créer user en DB. Stocker token dans flutter_secure_storage. Rediriger vers /home.	5h	Claude Code	Login/Register fonctionnel sur device

3B — Écrans core : Marché & Fiche action (S6–S7)

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P3-06	Écran Dashboard / Marché	Premier écran après login. Header : indices BRVM Composite, BRVM 30, Prestige (badges colorés). Section Top Hausses (5 actions) + Top Baisses (5 actions) : PriceChip avec variation %. Section Marché complet : liste scrollable triable. Pull-to-refresh. Skeleton loading.	6h	Claude Code	Dashboard affiché avec données réelles

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P3-07	Fiche action — partie gratuite	Écran /stock/:symbol. Header : nom, symbol, cours actuel, variation % (couleur). Section dividende : montant dernier dividende + rendement calculé. Section cours : graphique fl_chart 30 derniers jours (courbe). Badge secteur. Bouton 'Ajouter au portefeuille'.	6h	Claude Code	Fiche action gratuite complète
P3-08	Fiche action — partie Premium	Section bloquée si non Premium (afficher paywall avec CTA). Si Premium : onglet Santé Financière (graphiques barres CA + résultat net 3 ans), onglet Dividendes (historique tableau + prochain dividende + compte à rebours date détachement), onglet Liquidité (badge liquidité + volume moyen).	7h	Claude Code	Fiche action Premium avec paywall

3C — Fonctionnalités Premium (S7–S8)

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P3-09	Écran Portfolio	Liste lignes portfolio : nom action, quantité, PRU, cours actuel, gain/perte (€ et %). Total valorisation portefeuille. Total return (plus-value + dividendes reçus). Bouton ajouter ligne : bottom sheet avec recherche action + quantité + prix achat. Graphique camembert répartition.	7h	Claude Code	Portfolio avec PRU et total return
P3-10	Écran Alertes	Liste alertes actives : action, type (prix / dividende), seuil, statut. Toggle activer/désactiver. Bouton créer alerte : choisir action, type, seuil. FCM : afficher notification push reçue même app en arrière-plan (firebase_messaging onBackgroundMessage).	5h	Claude Code	Notification push reçue sur device
P3-11	Écran Comparateur sectoriel	Selector secteur (banques, télécoms, agro, etc.). Tableau comparatif : nom, PER, rendement %, variation 30j, badge liquidité. Tri par colonne. Premium only. Si non Premium : paywall.	5h	Claude Code	Comparateur trié par rendement
P3-12	Écran Simulateur	Slider montant mensuel (5 000 → 500 000 FCFA). Slider durée (1 → 20 ans). Slider rendement cible (5% → 20%). Résultat : capital final, dividendes cumulés, graphique projection fl_chart. Disclaimer légal obligatoire en bas. Accessible gratuit.	4h	Claude Code	Simulateur avec graphique
P3-13	Écran Abonnement / Paywall	Présentation plans Mensuel (2 000 FCFA) et Annuel (18 000 FCFA, économie mise en avant). Bouton Payer avec Wave → ouvrir URL Wave dans webview ou navigateur. Après retour : appel GET /auth/me pour vérifier upgrade. Afficher confirmation si Premium activé.	5h	Claude Code	Flux paiement Wave complet

3D — Finitions & qualité (S9)

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P3-14	Écran Profil	Email, plan (FREE/PREMIUM), date fin abonnement. Bouton déconnexion (Firebase signOut + clear secure storage). Bouton 'Passer Premium' si gratuit. Version app + lien conditions d'utilisation.	2h	Claude Code	Profil complet

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P3-15	Écran Onboarding	3 slides au premier lancement : 1) 'Toute la BRVM en 3 clics' 2) 'Suivez vos dividendes' 3) 'Commencez gratuitement'. Skip disponible. Stocké dans SharedPreferences (ne plus afficher après premier lancement).	3h	Claude Code	Onboarding affiché une seule fois
P3-16	Gestion erreurs & états vides	Chaque écran doit gérer : état loading (shimmer), état erreur réseau (widget retry), état vide (illustration + message). Pas d'écran blanc. Snackbar pour erreurs non-bloquantes.	4h	Claude Code	0 écran blanc en cas d'erreur
P3-17	Tests Flutter	Widget tests pour : DashboardScreen (mock API), FicheActionScreen (mock data), PortfolioScreen (calcul PRU). Integration test : flux login → dashboard → fiche action. flutter test --coverage.	4h	Claude Code	Tests Flutter passent
P3-18	Performance & optimisations	Activer HTTP caching sur Dio (ETag). ListView.builder sur toutes les listes longues. Compresser assets (images WebP). Vérifier taille APK release < 20MB. Profiler avec Flutter DevTools : pas de jank sur scroll.	3h	Claude Code	APK release < 20MB, scroll fluide

- CHECKPOINT — Critères de passage
- ✓ App installée et fonctionnelle sur device Android réel (pas seulement émulateur)

✓ Flux complet : inscription → dashboard → fiche action → paiement Wave → Premium activé

✓ Notifications push reçues sur device (alerte prix + dividende)

✓ 0 crash sur les 5 écrans principaux en utilisation normale

✓ flutter test : 0 échec

PHASE 4	LANCEMENT PLAY STORE	Bêta fermée · Monitoring · Feedback · Itération	Semaine 9
---------	----------------------	---	-----------

Objectif : publier sur le Play Store en bêta fermée (10-20 testeurs). Recueillir les premiers retours réels avant ouverture publique.

ID	Tâche	Détail & instructions	Durée	Outil IA	Livrable
P4-01	Préparer assets Play Store	Icône app 512x512px (PNG, pas de fond transparent). Feature graphic 1024x500px. Screenshots : 8 captures obligatoires (écrans clés : dashboard, fiche action, portfolio, alertes). Description courte (80 car.) + longue (4000 car.) en français. Catégorie : Finance.	4h	Midjourney + Manuel	Assets Play Store prêts
P4-02	Build APK release signé	Générer keystore : keytool -genkey -v -keystore akwaba.jks. Configurer build.gradle avec signing config. flutter build apk --release. Vérifier taille. Tester APK release sur device (pas debug).	3h	Claude (ici)	APK release signé et fonctionnel
P4-03	Créer compte Google Play Console	Frais uniques 25\$. Remplir profil développeur. Accepter accords développeur. Créer nouvelle app. Renseigner fiche store complète. Politique de confidentialité obligatoire (héberger sur domaine).	3h	Manuel	App créée sur Play Console
P4-04	Publier bêta fermée	Play Console → Tests → Bêta interne. Uploader APK/AAB. Ajouter 10-20 testeurs (emails). Publier. Délai de validation Google : 1-3 jours. Envoyer lien bêta aux testeurs.	2h	Manuel	App accessible aux bêta-testeurs
P4-05	Configurer monitoring	Sentry : intégrer sentry_flutter dans l'app (crash reports). UptimeRobot : monitorer https://api.akwaba-invest.com/health toutes les 5min. Alerte email si downtime. Better Stack optionnel pour logs centralisés.	3h	Claude (ici)	Alertes monitoring actives
P4-06	Plan de collecte feedback	Créer formulaire Google Forms (10 questions max). Envoyer aux 10 bêta-testeurs J+3 après installation. Questions : fonctionnalités manquantes, bugs rencontrés, willingness to pay, NPS. Deadline retours : 1 semaine.	2h	Manuel	Formulaire envoyé, retours collectés

CHECKPOINT — Critères de passage	✓ APK release installable depuis le Play Store (bêta interne) ✓ 10 testeurs ont installé et utilisé l'app au moins 1 session ✓ 0 crash critique rapporté dans Sentry les 48 premières heures ✓ Monitoring UptimeRobot actif — alerte testée ✓ Formulaire feedback retourné par au moins 5 testeurs
----------------------------------	--

Annexe A — Stack technique complète

Couche	Technologie	Version cible	Rôle
Mobile	Flutter	3.x stable	App Android
State mgmt	Riverpod	2.x	State management
Navigation	GoRouter	latest	Routing déclaratif
HTTP	Dio	5.x	Requêtes API + intercepteurs
Models	Freezed + json_serializable	latest	Immutabilité + sérialisation
Storage local	flutter_secure_storage	latest	Token Firebase sécurisé
Charts	fl_chart	0.6x	Graphiques cours + portfolio
Push	firebase_messaging	latest	Notifications FCM
Backend	FastAPI	0.110+	API REST
ORM	SQLModel	latest	Modèles + requêtes PostgreSQL
Migrations	Alembic	1.x	Versionning schéma DB
DB	PostgreSQL	15	Base de données principale
Auth	Firebase Auth	latest	Authentication
Push backend	firebase-admin	6.x	Envoi FCM depuis backend
PDF parsing	pdfplumber	0.9+	Extraction BOC
Serveur	Nginx	latest	Reverse proxy + SSL
Hébergement	Hetzner CX22	Ubuntu 22.04	VPS 4-5€/mois
Scheduler	Cron Linux natif	—	Parser BOC 18h quotidien
Paielements	Wave Business API	v2	Abonnements Premium
Monitoring	Sentry + UptimeRobot	—	Crash + uptime
CI/CD	GitHub Actions	—	Tests auto sur push

Annexe B — Budget estimatif mensuel

Poste	Coût mensuel	Notes
VPS Hetzner CX22	~4,50 €	PostgreSQL + FastAPI + Nginx inclus
Domaine (.com ou .app)	~1 €	Amorti sur 12 mois
Firebase	0 €	Free tier largement suffisant au départ
Wave Business	1% / transaction	Seulement sur abonnements encaissés
Sentry	0 €	Free tier 5 000 erreurs/mois
UptimeRobot	0 €	Free tier 50 moniteurs
GitHub	0 €	Free tier repo public
TOTAL fixe	~5,50 €/mois	Coût marginal quasi nul au démarrage